e050d9fe-844f-4daa-90a9-59883a48c0

<prev_user>

<request>

You are working on a task in the 'impierce/ssi-agent' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ssi-agent/ssi-agent (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-30T06:46:30.818+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
The caller of the API needs to know which fields they need to supply when creating a template with the dataModel for OpenBadges. How could this be solved? Does it make sense to offer another "read-only" endpoint which returns the "default schema" for certain data models or are there other options? What about leveraging the OpenAPI standard and adding a variant for the OpenBadges case or is that too complicating?
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<prev_assistant>

Now I have a good understanding of the codebase. This is a design/analysis question, so I'll provide a plan with options.

<plan>

Problem

API callers need to know which fields to supply when creating a template with `dataModel: "open_badges_3-0"`. Currently, the OpenBadges default schema properties (`achievement.name`, `achievement.criteria.narrative`) are silently merged server-side during `CreateTemplate` handling (see `merge_open_badges_defaults` in `agent_library/src/template/aggregate.rs:631`), but there's no way for a client to discover these requirements upfront.

Options

e merged schema).

Cons:
- Discovery is only possible *after* creating a template (or reading docs).
- Doesn't solve the "upfront discovery" problem.

Option D: Hybrid — Option A + OpenAPI examples

Add the read-only endpoint (Option A) **and** add an OpenBadges example to the `create_template` OpenAPI path (which is trivial with `utoipa`'s `examples(...)` macro).

Recommendation

Option A (or the hybrid Option D) is the best fit for this codebase because:

1. The building blocks already exist: `open_badges_default_schema_properties()` and `open_badges_default_required_keys()` are public and compose the full schema.

2. It follows the existing pattern of read-only query endpoints (`get_templates`, `get_template`).
3. It's extensible to ELM and future data models without changing the API contract shape.
4. It keeps the OpenAPI spec simple (no discriminator gymnastics).

Implementation sketch (Option A):
- Add a handler in `agent_api_http/src/v0/templates/mod.rs`:
  ```
  GET /templates/data-models/{data_model}/default-schema
  ```
- The handler matches on the `DataModel` enum variant and returns the composed default schema (or 404 for models without one).
- Register the route in the templates router and add it to the `TemplatesApi` OpenAPI struct.
- Optionally add an OpenBadges-specific example to the `create_template` path's `examples(...)`.

</plan>

<current>

<request>

You are working on a task in the 'impierce/ssi-agent' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ssi-agent/ssi-agent (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-30T06:54:05.927+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
Implement the plan:

Problem

API callers need to know which fields to supply when creating a template with `dataModel: "open_badges_3-0"`. Currently, the OpenBadges default schema properties (`achievement.name`, `achievement.criteria.narrative`) are silently merged server-side during `CreateTemplate` handling (see `merge_open_badges_defaults` in `agent_library/src/template/`

````
aggregate.rs:631`), but there's no way for a client to discover
these requirements upfront.

---

## Options

### Option A: Read-only endpoint `GET /templates/data-models/
{data_model}/default-schema` (Recommended)

Add a new endpoint that returns the default/base JSON Schema for
a given data model.

**How it works:**
- New route: `GET /v0/templates/data-models/open_badges_3-0/
default-schema`
- Returns the full JSON Schema that would be auto-merged
(properties, required keys) for that data model.
- For `w3c_vc_data_model_v1-1` (or models without a preset
schema), return an empty object schema `{"type": "object"}` or
404.
- The public functions `open_badges_default_schema_properties()`
and `open_badges_default_required_keys()` already exist and are
`pub`, so the endpoint can compose the full schema trivially.

**Pros:**
- Clean separation of concerns — discovery is decoupled from
creation.
- Easy for clients to call before building their request.
- Can be extended for future data models (ELM, etc.) without
touching the OpenAPI spec structure.
- Cacheable (the defaults are static).

**Cons:**
- One additional endpoint to maintain.

---

### Option B: Leverage OpenAPI `oneOf`/`discriminator` for the
request body

Modify `CreateTemplateEndpointRequest` in the OpenAPI spec to
use a discriminated union:

```yaml
requestBody:
  oneOf:
````
````

```
    - $ref: '#/components/schemas/CreateTemplateGeneric'
    - $ref: '#/components/schemas/CreateTemplateOpenBadges'
  discriminator:
    propertyName: dataModel
```

Where `CreateTemplateOpenBadges` explicitly lists the required `schema.properties` fields.

Pros:
- Self-documenting in the generated OpenAPI spec; SDK generators can produce typed request objects per variant.
- No extra endpoint needed.

Cons:
- Significantly complicates the OpenAPI schema — `utoipa` doesn't have first-class support for `discriminator` on request bodies, so you'd likely need manual YAML patches or a custom `ToSchema` impl.
- Mixes static documentation concerns with runtime behavior — the server still auto-merges, so the "required" annotation in OpenAPI is advisory only.
- Becomes combinatorially complex if more data models are added (each needs its own variant).
- Doesn't help clients that don't use code-generated SDKs.

Option C: Return the effective schema in the `CreateTemplate` response + document in examples

Instead of a separate endpoint, rely on:
1. The existing `POST /templates/create-template` response already returns the full `TemplateDto` including the merged `schema` — so the caller can see what was applied after the fact.
2. Enrich the OpenAPI examples (already partially done) with an OpenBadges-specific example showing required fields.

Pros:
- Zero new endpoints or schema changes.
- Already partially implemented (the response includes the merged schema).

Cons:
- Discovery is only possible *after* creating a template (or reading docs).

````
- Doesn't solve the "upfront discovery" problem.

---

### Option D: Hybrid — Option A + OpenAPI examples

Add the read-only endpoint (Option A) **and** add an OpenBadges
example to the `create_template` OpenAPI path (which is trivial
with `utoipa`'s `examples(...)` macro).

---

## Recommendation

**Option A** (or the hybrid Option D) is the best fit for this
codebase because:

1. The building blocks already exist:
`open_badges_default_schema_properties()` and
`open_badges_default_required_keys()` are public and compose the
full schema.
2. It follows the existing pattern of read-only query endpoints
(`get_templates`, `get_template`).
3. It's extensible to ELM and future data models without
changing the API contract shape.
4. It keeps the OpenAPI spec simple (no discriminator
gymnastics).

**Implementation sketch (Option A):**
- Add a handler in `agent_api_http/src/v0/templates/mod.rs`:
  ```
  GET /templates/data-models/{data_model}/default-schema
  ```
- The handler matches on the `DataModel` enum variant and
returns the composed default schema (or 404 for models without
one).
- Register the route in the templates router and add it to the
`TemplatesApi` OpenAPI struct.
- Optionally add an OpenBadges-specific example to the
`create_template` path's `examples(...)`.
</problem_statement>
````

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

```
**If the task requires code changes:**

**If the task is a question or only requires creating a plan or
analysis:** respond directly without making code changes.
```